

Laboratory work #1: ERD diagram for a Airport management system

Student Name: Kassymkozha Yergali

Course: Databases

Topic: Entity-Relationship Diagram & Database Design for Airport Management System

1. System Overview

The primary goal of this database system is to manage international airport operations, including airport and airline information, scheduled and actual flights, passenger records, flight bookings, booking changes, boarding passes, baggage registration, baggage inspections, and passenger security checks.

The system organizes airport-related information in a structured relational database. Primary keys uniquely identify records, while foreign keys establish relationships between airports, airlines, flights, passengers, bookings, and other operational entities. Unique and check constraints are also used to prevent duplicate or invalid data.

This structure helps maintain data integrity and provides efficient management of passenger reservations, flight operations, boarding procedures, baggage processing, and airport security activities.

2. Entity-Relationship Diagram (ERD)

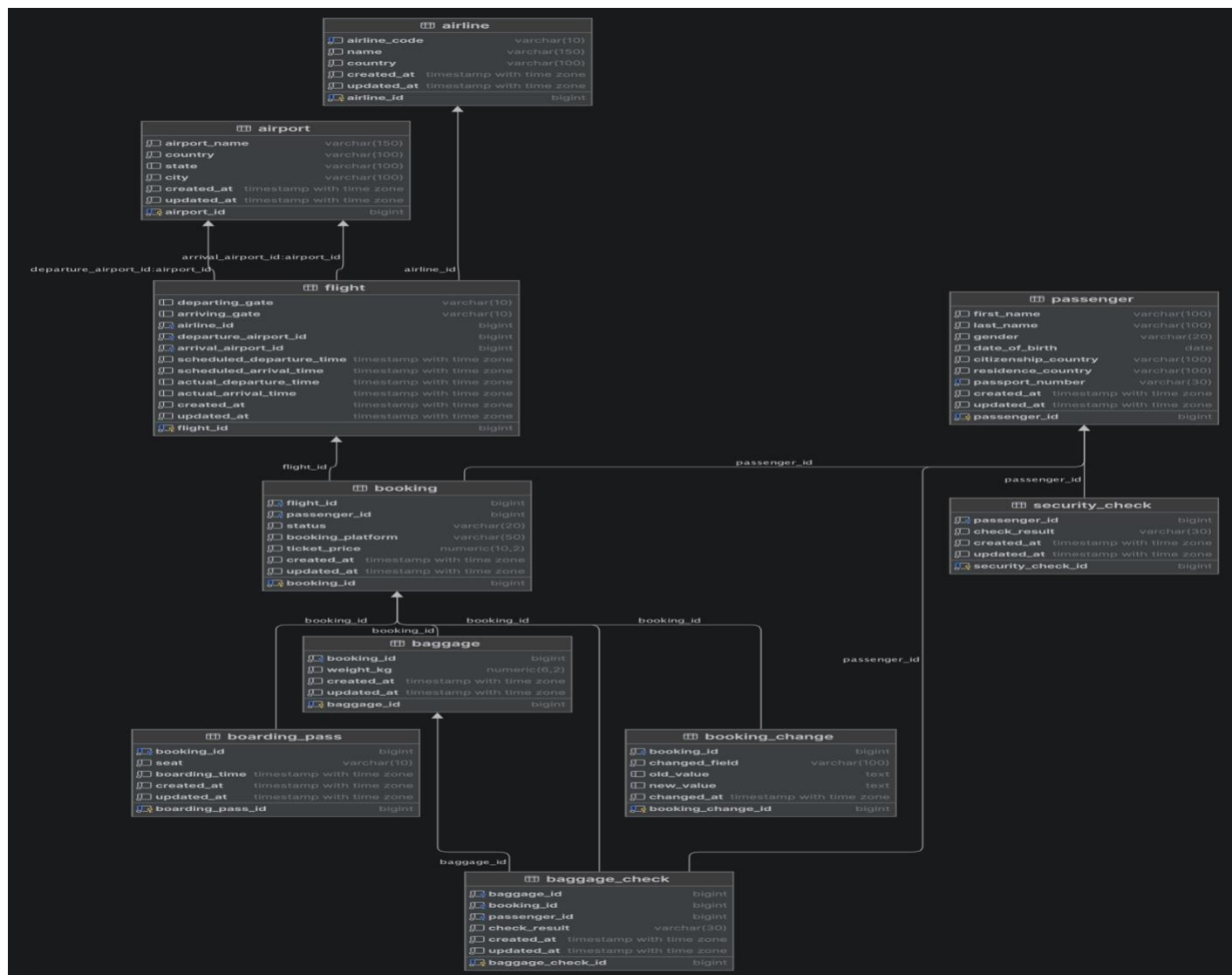


Diagram Legend

Symbol / Notation	Name	Description
PK	Primary Key	Unique identifier for each record in an entity table.
FK	Foreign Key	Attribute referencing the Primary Key of another table to establish relationships.
1 : N	One-to-Many	One record in the parent table relates to multiple records in the child table.
N : 1	Many-to-One	Multiple records in one table relate to one record in another table.
1 : 1	One-to-One	One record in the parent table relates to exactly one record in the child table.
N : M	Many-to-Many	Many records in one table can be related to many records in another table through an intermediate table.

3. Entities, Attributes, and Data Types

The system consists of 9 primary entities, each containing attributes and primary or foreign keys.

Airport

Stores information about airports, including their names and geographical locations.

Attributes: airport_id: BIGINT [PK], airport_name: VARCHAR(150), country: VARCHAR(100), state: VARCHAR(100), city: VARCHAR(100), created_at: TIMESTAMP WITH TIME ZONE, updated_at: TIMESTAMP WITH TIME ZONE.

Airline

Stores information about airline companies that operate flights.

Attributes: airline_id: BIGINT [PK], airline_code: VARCHAR(10) [UQ], name: VARCHAR(150), country: VARCHAR(100), created_at: TIMESTAMP WITH TIME ZONE, updated_at: TIMESTAMP WITH TIME ZONE.

Flight

Stores scheduled and actual flight information. Each flight is operated by one airline and connects a departure airport with an arrival airport.

Attributes: flight_id: BIGINT [PK], departing_gate: VARCHAR(10), arriving_gate: VARCHAR(10), airline_id: BIGINT [FK], departure_airport_id: BIGINT [FK], arrival_airport_id: BIGINT [FK], scheduled_departure_time: TIMESTAMP WITH TIME ZONE, scheduled_arrival_time: TIMESTAMP WITH TIME ZONE, actual_departure_time: TIMESTAMP WITH TIME ZONE, actual_arrival_time: TIMESTAMP WITH TIME ZONE, created_at: TIMESTAMP WITH TIME ZONE, updated_at: TIMESTAMP WITH TIME ZONE.

Passenger

Stores passengers' personal, identification, citizenship, and residence information.

Attributes: passenger_id: BIGINT [PK], first_name: VARCHAR(100), last_name: VARCHAR(100), gender: VARCHAR(20), date_of_birth: DATE, citizenship_country: VARCHAR(100), residence_country: VARCHAR(100), passport_number: VARCHAR(30) [UQ], created_at: TIMESTAMP WITH TIME ZONE, updated_at: TIMESTAMP WITH TIME ZONE.

Booking

Stores flight reservations made by passengers. Each booking connects one passenger with one flight.

Attributes: booking_id: BIGINT [PK], flight_id: BIGINT [FK], passenger_id: BIGINT [FK], status: VARCHAR(20), booking_platform: VARCHAR(50), ticket_price: NUMERIC(10,2), created_at: TIMESTAMP WITH TIME ZONE, updated_at: TIMESTAMP WITH TIME ZONE.

BookingChange

Stores the history of changes made to bookings, including the modified field and its previous and new values.

Attributes: booking_change_id: BIGINT [PK], booking_id: BIGINT [FK], changed_field: VARCHAR(100), old_value: TEXT, new_value: TEXT, changed_at: TIMESTAMP WITH TIME ZONE.

BoardingPass

Stores information about boarding passes issued for bookings, including assigned seats and boarding times.

Attributes: boarding_pass_id: BIGINT [PK], booking_id: BIGINT [FK, UQ], seat: VARCHAR(10), boarding_time: TIMESTAMP WITH TIME ZONE, created_at: TIMESTAMP WITH TIME ZONE, updated_at: TIMESTAMP WITH TIME ZONE.

Baggage

Stores information about baggage items registered under passenger bookings.

Attributes: baggage_id: BIGINT [PK], booking_id: BIGINT [FK], weight_kg: NUMERIC(6,2), created_at: TIMESTAMP WITH TIME ZONE, updated_at: TIMESTAMP WITH TIME ZONE.

BaggageCheck

Stores information about baggage inspection results. Each record identifies the checked baggage, its booking, and its passenger.

Attributes: baggage_check_id: BIGINT [PK], baggage_id: BIGINT [FK], booking_id: BIGINT [FK], passenger_id: BIGINT [FK], check_result: VARCHAR(30), created_at: TIMESTAMP WITH TIME ZONE, updated_at: TIMESTAMP WITH TIME ZONE.

SecurityCheck

Stores the results of security checks performed on passengers.

Attributes: security_check_id: BIGINT [PK], passenger_id: BIGINT [FK], check_result: VARCHAR(30), created_at: TIMESTAMP WITH TIME ZONE, updated_at: TIMESTAMP WITH TIME ZONE.

4. Relationships and Constraints

This database supports the management of international airport operations, including airports, airlines, flights, passengers, bookings, boarding passes, baggage registration, baggage checks, security checks, and booking changes.

The tables establish several types of relationships.

One-to-Many (1:N)

A one-to-many relationship means that one record in one table can be associated with multiple records in another table.

Examples include:

- **Airline to Flight:** One airline can operate multiple flights, while each flight belongs to one airline.
- **Airport to Flight:** One airport can be used as the departure airport for multiple flights.
- **Airport to Flight:** One airport can also be used as the arrival airport for multiple flights.
- **Flight to Booking:** One flight can have multiple passenger bookings, while each booking belongs to one flight.
- **Passenger to Booking:** One passenger can make multiple bookings, while each booking belongs to one passenger.
- **Passenger to SecurityCheck:** One passenger can have multiple security checks over time.
- **Booking to Baggage:** One booking can include multiple baggage items.
- **Booking to BookingChange:** One booking can have multiple change records.
- **Baggage to BaggageCheck:** One baggage item can have multiple inspection records.
- **Booking to BaggageCheck:** One booking can be connected to multiple baggage checks.
- **Passenger to BaggageCheck:** One passenger can be connected to multiple baggage checks.

Many-to-One (N:1)

A many-to-one relationship means that multiple records in one table can reference one record in another table.

Examples include:

- Multiple flights can reference one airline through `airline_id`.
- Multiple flights can reference one departure airport through `departure_airport_id`.
- Multiple flights can reference one arrival airport through `arrival_airport_id`.
- Multiple bookings can reference one flight through `flight_id`.
- Multiple bookings can reference one passenger through `passenger_id`.
- Multiple baggage records can reference one booking through `booking_id`.
- Multiple booking changes can reference one booking through `booking_id`.

- Multiple baggage checks can reference one baggage item, booking, or passenger through `baggage_id`, `booking_id`, and `passenger_id`.
- Multiple security checks can reference one passenger through `passenger_id`.

One-to-One (1:1)

A one-to-one relationship means that one record in one table can be associated with only one record in another table.

The relationship between **Booking** and **BoardingPass** is represented as an optional one-to-one relationship. Each boarding pass belongs to exactly one booking, and one booking can have no boarding pass or one boarding pass.

This relationship is implemented using `booking_id` as a foreign key and a unique value in the `boarding_pass` table. A booking may initially have no boarding pass because it can be issued later during check-in.

Many-to-Many (N:M)

A many-to-many relationship occurs when multiple records in one table can be associated with multiple records in another table through an intermediate table.

In this database, **Passenger** and **Flight** form a many-to-many relationship through the **Booking** table.

A passenger can book multiple flights, while one flight can have bookings from multiple passengers. The `booking` table resolves this relationship by storing `passenger_id` and `flight_id`.

Therefore, the many-to-many relationship is implemented as two one-to-many relationships:

- Passenger to Booking: 1:N
- Flight to Booking: 1:N

5. Primary and Foreign Key Constraints

Primary keys uniquely identify records within each table.

The following primary keys are used:

- `airport_id` uniquely identifies an airport.
- `airline_id` uniquely identifies an airline.
- `flight_id` uniquely identifies a flight.
- `passenger_id` uniquely identifies a passenger.
- `booking_id` uniquely identifies a booking.

- `booking_change_id` uniquely identifies a booking change.
- `boarding_pass_id` uniquely identifies a boarding pass.
- `baggage_id` uniquely identifies a baggage item.
- `baggage_check_id` uniquely identifies a baggage check.
- `security_check_id` uniquely identifies a security check.

Foreign keys establish relationships between tables.

The `flight` table uses `airline_id` to connect a flight with an airline. It also uses `departure_airport_id` and `arrival_airport_id` to connect a flight with its departure and arrival airports.

The `booking` table uses `flight_id` and `passenger_id` to connect each booking with one flight and one passenger.

The `booking_change`, `boarding_pass`, and `baggage` tables use `booking_id` to connect their records with the appropriate booking.

The `baggage_check` table uses `baggage_id`, `booking_id`, and `passenger_id` to connect an inspection result with the checked baggage item, its booking, and its passenger.

The `security_check` table uses `passenger_id` to connect each security check with the appropriate passenger.

The database also uses unique constraints:

- `airline_code` must be unique for every airline.
- `passport_number` must be unique for every passenger.
- `booking_id` in the `boarding_pass` table must be unique to ensure that one booking has no more than one boarding pass.

Additional check constraints maintain valid data:

- The departure airport and arrival airport of a flight must be different.
- The scheduled arrival time must be later than the scheduled departure time.
- The ticket price cannot be negative.
- Baggage weight must be greater than zero.
- Booking status must contain an allowed value such as `pending`, `confirmed`, `cancelled`, or `checked_in`.

These primary key, foreign key, unique, and check constraints maintain referential integrity and ensure that relationships between airport system entities remain consistent.

6. Conclusion

The International Airport Database provides a structured approach to managing airport and flight operations.

The ER diagram organizes information into 10

entities: **Airport**, **Airline**, **Flight**, **Passenger**, **Booking**, **BookingChange**, **BoardingPass**, **Baggage**, **BaggageCheck**, and **SecurityCheck**.

Primary keys provide unique identification for every record, while foreign keys connect related entities. The database supports one-to-many, many-to-one, one-to-one, and many-to-many relationships.

The booking table plays an important role because it connects passengers with flights. It also provides the main connection for boarding passes, baggage records, baggage inspections, and booking changes.

Overall, the database design provides an effective foundation for managing airports, airlines, flights, passenger reservations, boarding passes, baggage processing, and security checks while maintaining data integrity and organized data management.